Graph Applications

CSE 4303: Data Structures
Lecture 8

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

December 31, 2025

Recap: Articulation Points & Bridges

Before diving into the algorithms, let's define the structural weaknesses in a graph.

Articulation Point (Cut Vertex)

A vertex v is an **Articulation Point** if removing v (and its incident edges) increases the number of connected components in the graph.

👉 **Implication:** These are single points of failure in a network.

Bridge (Cut Edge)

An edge (u, v) is a **Bridge** if removing it increases the number of connected components.

👉 **Relation:** A bridge always connects two articulation points (unless one end is a leaf node).

Goal: Find all such points/edges efficiently in $O(V + E)$ using DFS, rather than the naive $O(V(V + E))$ approach.

Finding Articulation Points: The Algorithm & Reasoning I

The Core Idea: DFS Tree Properties

- When we run DFS, we build a **DFS Tree**.
- Edges in the graph fall into two categories:
  1. **Tree Edges**: Edges we traverse to discover new nodes.
  2. **Back Edges**: Edges connecting a node to an already visited ancestor.
- **Key Insight**: Back edges form cycles. They provide "alternative paths" in the graph. If a node's subtree has a back edge to an ancestor, removing that node won't disconnect the subtree.

Finding Articulation Points: The Algorithm & Reasoning II

Step 1: Initialization

- We maintain a global `timer` and two arrays for every node u:
  - $d[u]$ (or $tin[u]$): **Discovery Time**. When did we first enter this node?
  - $low[u]$: **Low-Link Value**. The lowest discovery time reachable from u (including itself) via the subtree or a single back-edge.
- *Reasoning*: $d[u]$ identifies the node's position in the DFS tree. $low[u]$ tells us if there is a "secret path" connecting u's descendants back to u's ancestors.

Finding Articulation Points: The Algorithm & Reasoning III

Step 2: Processing Neighbors (The Logic) For a node u and its neighbor v:

👉 **Case A: v is the Parent of u**

- **Action**: Ignore this edge.
- **Reasoning**: Going back to the node we just came from is trivial and doesn't count as finding a cycle (back-edge) in the context of undirected graphs.

👉 **Case B: v is already Visited (Back Edge)**

- **Action**: $low[u] = \min(low[u], d[v])$
- **Reasoning**: We found a back-edge! Node u connects to an ancestor v. This means u is part of a cycle. We update $low[u]$ to reflect that u can reach this older node v.

Finding Articulation Points: The Algorithm & Reasoning IV

👉 Case C: v is Unvisited (Tree Edge)

- **Action 1**: Recursively call DFS(v).
- **Action 2**: On return, update $low[u] = \min(low[u], low[v])$.
- **Reasoning**: After v explores its entire subtree, it returns the "highest" point it could reach. If v found a way to reach an ancestor of u, $low[v]$ will be small. We propagate this reachability up to u.

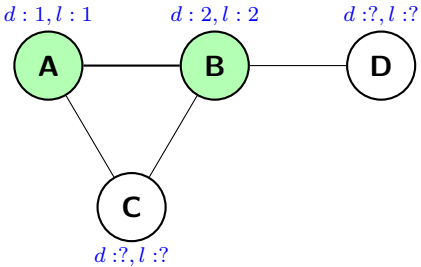
- **Action 3 (CRITICAL CHECK)**:
  If $low[v] \geq d[u]$ (and $u \neq$ root), then u **is a Cutpoint**.
- **Reasoning**:
  - ⋆ $low[v] \geq d[u]$ means the subtree rooted at v has **NO back-edge** to any ancestor of u.
  - ⋆ The "lowest" node v can reach is u itself.
  - ⋆ Therefore, all paths from v to the rest of the graph **must go through** u. If we remove u, v becomes disconnected.

Finding Articulation Points: The Algorithm & Reasoning V

Step 3: The Special Root Case

- 👉 The condition $low[v] \geq d[u]$ is always true for the root (since it has the smallest d value, 0 or 1).
- 👉 **Rule**: The root is a cutpoint if and only if it has **more than 1 child** in the DFS tree.
- 👉 **Reasoning**:
  - In a DFS tree, if the root has two children, say X and Y, it implies there is **no edge** between the subtree of X and the subtree of Y.
  - If there were an edge, DFS would have moved from X to Y directly, making Y a descendant of X, not a separate child of Root.
  - Therefore, removing Root disconnects X and Y.

Articulation Point Animation



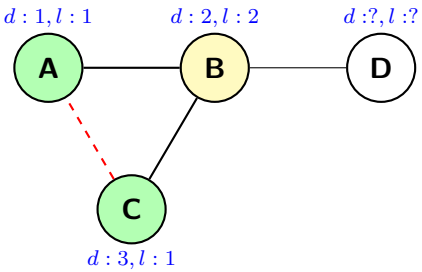
DFS Traversal Log:

1. **Start DFS at A**: $d[A] = 1, low[A] = 1$.
2. **Move to B**: $d[B] = 2, low[B] = 2$.

Graph Structure

- **Cycle (A-B-C)**: Nodes can reach back to ancestors. low values decrease.
- **Tail (D)**: Node D relies entirely on B. Removing B disconnects D.

Articulation Point Animation



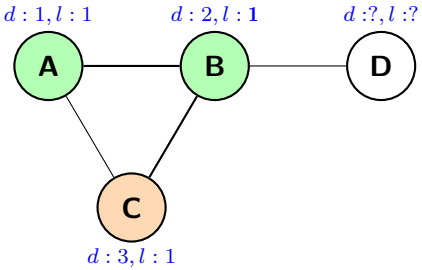
DFS Traversal Log:

3. Move B → C: $d[C] = 3, low[C] = 3$.
4. C sees A (Visited): Back-edge!
Update $low[C] = \min(3, d[A] = 1) = \mathbf{1}$.

Graph Structure

- 👉 **Cycle (A-B-C)**: Nodes can reach back to ancestors. low values decrease.
- 👉 **Tail (D)**: Node D relies entirely on B. Removing B disconnects D.

Articulation Point Animation



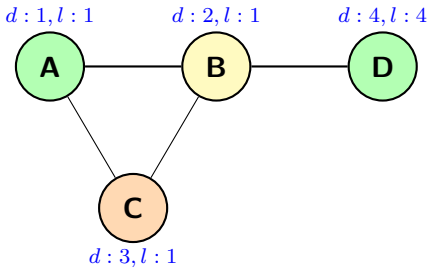
DFS Traversal Log:

5. Backtrack C → B:
Update $low[B] = \min(low[B], low[C]) = 1$.
Check AP: Child C has $low[C](1) < d[B](2)$.
Condition Fails (B is NOT AP for C).

Graph Structure

- 👉 **Cycle (A-B-C)**: Nodes can reach back to ancestors. low values decrease.
- 👉 **Tail (D)**: Node D relies entirely on B. Removing B disconnects D.

Articulation Point Animation



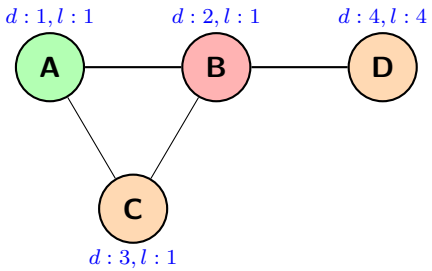
DFS Traversal Log:

6. Move B $\rightarrow$ D: $d[D] = 4, low[D] = 4$.
D has no other neighbors.

Graph Structure

- **Cycle (A-B-C)**: Nodes can reach back to ancestors. low values decrease.
- **Tail (D)**: Node D relies entirely on B. Removing B disconnects D.

Articulation Point Animation



DFS Traversal Log:

7. Backtrack D → B:
Check AP: Child D has $low[D](4) \geq d[B](2)$.
TRUE! B is an Articulation Point.

Graph Structure

- 👉 **Cycle (A-B-C)**: Nodes can reach back to ancestors. low values decrease.
- 👉 **Tail (D)**: Node D relies entirely on B. Removing B disconnects D.

Complexity Analysis: Finding Articulation Points I

Time Complexity: $O(V + E)$

The algorithm relies on a single **Depth First Search (DFS)** traversal.

- 👉 **Vertex Visits**: We visit every vertex u exactly once to initialize and process. $\rightarrow O(V)$
- 👉 **Edge Visits**: For every directed edge (u, v) (or undirected edge viewed twice), we perform constant time checks (Compare d and low, update min). $\rightarrow O(E)$

Total: $O(V + E)$, which is asymptotically optimal since we must read the graph at least once.

Complexity Analysis: Finding Articulation Points II

Space Complexity: $O(V)$

We use auxiliary space proportional to the number of vertices:

- 👉 **Arrays**: We store `discovery[V]`, `low[V]`, and `parent[V]` arrays. $\rightarrow O(V)$
- 👉 **Recursion Stack**: In the worst case (a skewed line graph), the DFS recursion depth can reach V. $\rightarrow O(V)$

Note: The Adjacency List used to store the graph takes $O(V + E)$, but that is considered input space.

13.9 Finding Bridges (Cut Edges) I

Definition

An edge (u, v) is a **Bridge** if removing it increases the number of connected components in the graph.

👉 Unlike Articulation Points, this refers to the *connection* itself, not the node.

The DFS Condition: In the DFS tree, an edge (u, v) (where v is a child of u) is a bridge if and only if:

$$\mathbf{low[v] > d[u]}$$

Reasoning for Strict Inequality ($>$):

13.9 Finding Bridges (Cut Edges) II

- **If $low[v] \leq d[u]$:** The subtree rooted at v has a back-edge to u or an ancestor of u. A cycle exists. Removing (u, v) does not disconnect v.
- **If $low[v] > d[u]$:** The "lowest" node v can reach is itself (or nodes below it). It has **no way back** to u or above. The edge (u, v) is the *only* path; hence, it is a Bridge.

Algorithm Steps (Bridge Finding) I

The algorithm is almost identical to Articulation Points, but simpler (no special root case).

Step 1: Initialization

- 👉 Set `timer = 0`.
- 👉 Initialize $d[]$ and $low[]$ arrays.

Algorithm Steps (Bridge Finding) II

Step 2: DFS Traversal For a node u and neighbor v:

👉 **Case A: v is Parent of u**

- Ignore. We don't use the direct path back to parent to update low-link.

👉 **Case B: v is Visited (Back Edge)**

- **Update:** $low[u] = \min(low[u], d[v])$.
- *Logic:* We found a cycle. u connects to an ancestor v.

Algorithm Steps (Bridge Finding) III

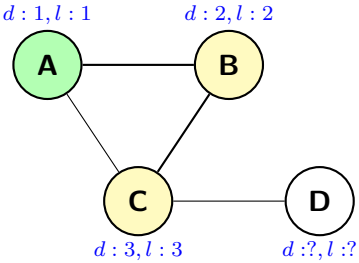
👉 **Case C: v is Unvisited (Tree Edge)**

- Recursively call DFS(v).
- **Update on Return:** $low[u] = \min(low[u], low[v])$.
- **Check Bridge Condition:**

$$\text{IF } low[v] > d[u] \implies (u, v) \text{ is a BRIDGE}$$

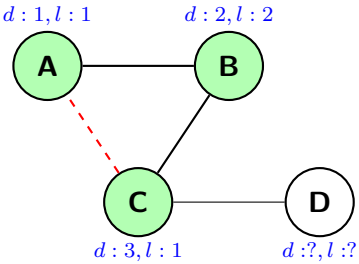
- *Logic:* After exploring all of v's descendants, if $low[v]$ is still greater than $d[u]$, it implies no back-edge goes above u. The connection is critical.

Bridge Finding Animation



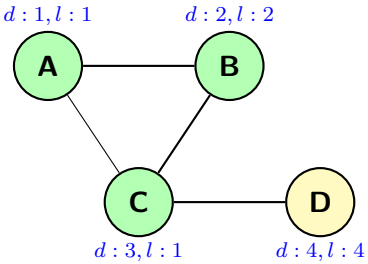
Log:
1. Traverse A $\rightarrow$ B $\rightarrow$ C:
$d[A] = 1, d[B] = 2, d[C] = 3.$
(All low values = d values initially).

Bridge Finding Animation



Log:
2. C sees A (Visited):
Back-Edge detected!
Update $low[C] = \min(3, d[A] = 1) = \mathbf{1}$.

Bridge Finding Animation



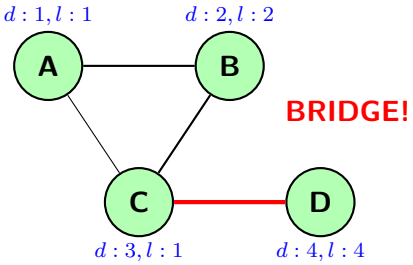
Log:

3. C moves to D:

$d[D] = 4, low[D] = 4.$

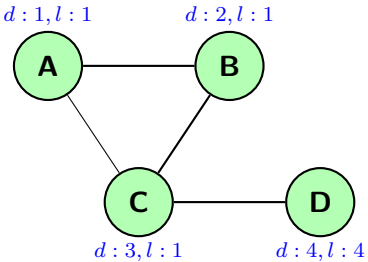
D has no other neighbors (Leaf).

Bridge Finding Animation



Log:
4. Backtrack D → C:
Check: $low[D](4) > d[C](3)$?
TRUE! The edge (C, D) is a **Bridge**.

Bridge Finding Animation



Log:
5. Backtrack C → B:
$low[B] = \min(2, low[C] = 1) = 1.$
Check: $low[C](1) > d[B](2)$?
FALSE. (B, C) is NOT a Bridge.

Complexity Analysis: Bridge Finding

Time Complexity: $O(V + E)$

Just like finding Articulation Points, this algorithm is a direct modification of DFS.

- We visit every node once.
- We traverse every edge exactly twice (once from each end) in an undirected graph.
- All updates (min operations) and checks ($>$) are $O(1)$.

Space Complexity: $O(V)$

- **DFS Stack:** Up to $O(V)$ depth in worst case.
- **Arrays:** $d[]$, $low[]$, $parent[]$ each take size V.

Summary: This is the most efficient known algorithm for finding bridges, far superior to the naive $O(E(V + E))$ approach of removing every edge and checking connectivity.

13.7 Topological Sorting

Definition

Topological sorting is a linear ordering of vertices in a directed graph such that for every directed edge $u \to v$, vertex u comes before v in the ordering.

Key Constraints:

- 👉 The graph must be a **DAG** (Directed Acyclic Graph).
- 👉 If the graph contains a cycle, a topological sort is **impossible**.

Real-World Analogy:

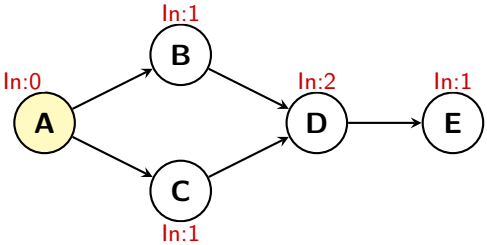
- 👉 **Prerequisites:** You cannot take "Advanced Data Structures" (v) before taking "Intro to C" (u).
- 👉 **Assembly:** You must put on socks (u) before shoes (v).

Algorithm Steps (Source Removal Method)

This approach uses **In-Degrees** to determine dependency.

1. **Calculate In-Degree**: Count the number of incoming edges for every node in the graph.
2. **Initialize Queue**: Add all nodes with `In-Degree = 0` to a Queue. (These are nodes with no dependencies).
3. **Process Loop** (While Queue is not empty):
   - Dequeue a node u.
   - Add u to the **Sorted Output List**.
   - For every neighbor v of u (effectively removing edge $u \to v$):
     - Decrement the In-Degree of v by 1.
     - **If** In-Degree of v becomes 0 $\to$ Enqueue v.
4. **Completion**: If the Sorted List contains all nodes, we are done. If not, the graph has a cycle.

Topological Sort Animation



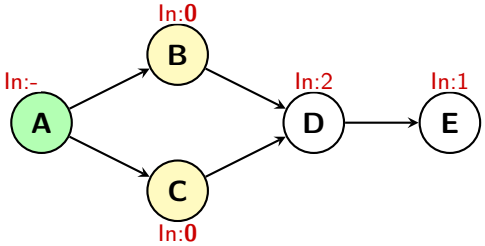
Log:
Step 1: Initialize.
Calc In-degrees.
A has 0 in-degree → Enqueue.

Queue:

A

Sorted Output List:
`{ (Empty) }`

Topological Sort Animation



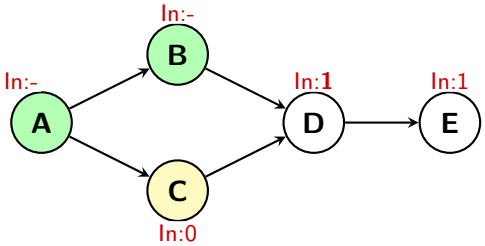
Log:
Step 2: Process A.
Add A to Output.
Reduce neighbors B & C.
B, C become 0 → Enqueue.

Queue:

B	C

Sorted Output List:
`{ A }`

Topological Sort Animation



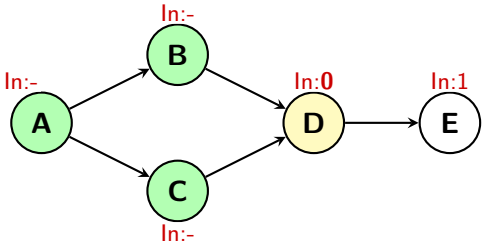
Log:
Step 3: Process B.
Add B to Output.
Reduce neighbor D ($2 \to 1$).
D is not yet 0.

Queue:

C

Sorted Output List:
`{ A, B }`

Topological Sort Animation



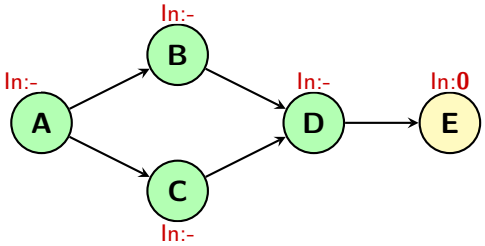
Log:
Step 4: Process C.
Add C to Output.
Reduce neighbor D ($1 \to 0$).
D becomes 0 $\to$ Enqueue.

Queue:

D

Sorted Output List:
`{ A, B, C }`

Topological Sort Animation



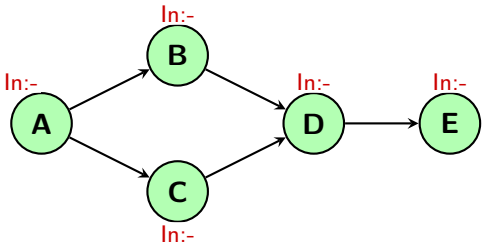
Log:
Step 5: Process D.
Add D to Output.
Reduce neighbor E ($1 \to 0$).
E becomes 0 $\to$ Enqueue.

Queue:

E

Sorted Output List:
`{ A, B, C, D }`

Topological Sort Animation



Log:
Step 6: Process E.
Add E to Output.
Queue Empty → Done.

Queue:

Empty

Sorted Output List:
`{ A, B, C, D, E }`

Complexity Analysis: Time Complexity $O(V + E)$ I

The efficiency comes from avoiding repeated scans of the node list.

1. **Step 1: Calculate In-Degrees** $O(V + E)$
   - We iterate through every node (V) and every outgoing edge (E) once to build the initial count.
2. **Step 2: Initial Discovery (Static Scan)** $O(V)$
   - We scan the In-Degree array **once** to find all nodes starting with 0 dependencies and add them to the Queue.

Complexity Analysis: Time Complexity $O(V + E)$ II

3. **Step 3: Processing Loop (Dynamic Update)** $O(E)$
   - **The mechanism:** We do **not** rescan all nodes to find the next available task.
   - Instead, we find them **dynamically**:
     - ⋆ When processing node u, we visit only its neighbors v.
     - ⋆ We decrement `InDegree[v]`.
     - ⋆ **Immediate Check:** If `InDegree[v]` drops to 0, we enqueue v immediately.
   - Since every edge is visited exactly once for a decrement operation, this entire phase takes linear time relative to edges.

Total Time: $O(V + E) + O(V) + O(E) \approx \mathbf{O(V + E)}$

Complexity Analysis: Space & Correctness I

Space Complexity: $O(V)$

We need auxiliary storage proportional to the number of vertices:

- 👉 **In-Degree Array:** Stores an integer count for all V nodes $\rightarrow O(V)$.
- 👉 **Queue:** In the worst case (e.g., a graph with no edges), all V nodes are stored in the queue simultaneously $\rightarrow O(V)$.
- 👉 **Sorted Output List:** Stores the result $\rightarrow O(V)$.

Note: The graph storage itself (Adjacency List) takes $O(V + E)$, but that is input storage.

Complexity Analysis: Space & Correctness II

Cycle Detection (Correctness)

Topological Sort doubles as a Cycle Detection algorithm.

- 👉 We maintain a counter of processed nodes.
- 👉 **Logic:** If `count < V` when the Queue becomes empty, it implies a **Cycle** exists.
- 👉 **Why?** Nodes in a cycle never reach In-Degree 0, so they never enter the Queue.

🤝 Reference & Q&A

Reference: Data Structures c Using 2nd Edition, Reema Thareja

- 👉 **Chapter 13**:Topological Sort
- 👉 **Articulation Point**: https://cp-algorithms.com/graph/cutpoints.html
- 👉 **Bridge Edges**: https://cp-algorithms.com/graph/bridge-searching.html

Thank You!

👋 Thank you for your time and attention. 👋

🗨 Any questions or clarifications. 💭